

BikeStores SQL Analytics Assessment

I. Basic Level (Questions 1–10)

Focus: INNER JOIN, LEFT JOIN, Basic Aggregations, and Simple Filtering.

1. **Product Catalog Details:** Write a query to list all products along with their respective brand name and category name. Sort the output alphabetically by category, then brand.
2. **Customer Order History:** Retrieve a list of all orders showing the order_id, order_date, and the full name of the customer (first_name and last_name) who placed it.
3. **Store Staff Assignment:** Display a list of all staff members (first_name, last_name) alongside the name of the store they are currently working at.
4. **Out-of-Stock Items:** Find all products that currently have a quantity of 0 or are completely missing from the stock records for a specific store (e.g., 'Santa Cruz Bikes').
5. **Revenue per Order Item:** Write a query to calculate the net revenue for every item in sales.order_items. Formula: Quantity * List Price * (1 - Discount).
6. **Top Priced Products:** Identify all products that have a list_price greater than the overall average list price of all products in the database.
7. **Active Customers:** Extract a list of unique customers who have placed at least one order where the order_status is completed (4).
8. **Manager Identification:** List the names of all staff members who are also managers (i.e., their staff_id appears in someone else's manager_id column).
9. **State-wise Sales Hubs:** Find all stores located in states where more than 50 customers reside.
10. **Brand Variety:** Identify which product categories contain products from more than 3 different brands.

II. Intermediate Level (Questions 11–20)

Focus: Multiple Joins, Common Table Expressions (CTEs), Subqueries, and Aggregations with HAVING.

11. **Order Value Breakdown (CTE):** Create a CTE that calculates the total order value (accounting for quantity, list price, and discount) for each order_id. Use it to find the average order value across the entire company.
12. **High-Value Customers:** Using a subquery or CTE, find the top 5 customers who have spent the most money across all their completed orders.
13. **Staff vs. Manager Hierarchy:** Write a query using a self-join to display every staff member's name along with their direct manager's name. Include staff who do not have a manager.
14. **Dead Inventory Detection:** Identify brands that have products in the production.products table but have never been sold in any order.
15. **Store Performance Summary:** For each store, calculate the total number of orders handled, total items sold, and total revenue generated.
16. **Delayed Shipments Analysis:** Find orders that took longer to ship than expected (shipped_date > required_date). Show the order details, the customer's email, and how many days it was delayed.
17. **Most Popular Category per Store:** Write a query to find which product category has the highest number of units sold for each individual store.
18. **Stock Shortage Warning:** Find products where the total stock across all stores is less than the total quantity of that product ordered in the last 30 days of the data history.
19. **Customer Cohort by Year:** Group customers by the model_year of the products they purchased to see if customers buying older models generate less revenue than those buying newer models.
20. **Underperforming Staff:** Identify staff members who have processed fewer orders than the average number of orders processed by all staff members within their same store.

III. Advanced Level (Questions 21–30)

Focus: Window Functions (ROW_NUMBER, RANK, LEAD, LAG), Running Totals, and Advanced Business Logic.

21. **Product Price Ranking:** Within each category, rank products by their list_price in descending order using DENSE_RANK(). Show only the top 3 most expensive products per category.
22. **Monthly Running Total Revenue:** Calculate the running total of sales revenue month-over-month for the fiscal year 2017.
23. **Customer Purchase Gap (LEAD/LAG):** For customers who have placed multiple orders, find the number of days that elapsed between their first order and their second order.
24. **Store Revenue Contribution:** For every order, display the order_id, store_id, order value, and the percentage contribution of that order to that specific store's total historical revenue.
25. **Staff Month-over-Month Growth:** Find the total sales revenue generated by each staff member per month, and use LAG() to show their revenue growth or decline compared to the previous month.
26. **Recursive Organization Chart:** Write a recursive CTE to display the entire staff management hierarchy, starting from the top manager down to the lowest-ranked staff, showing their hierarchy level (e.g., Level 1, Level 2).
27. **Inventory Velocity:** Calculate how many units of each product are currently in stock versus how many units have been sold. Rank products within their category by their "sales-to-stock" ratio.
28. **Repeated Product Purchases:** Identify customers who have ordered the exact same product_id on more than one separate occasion (different order_dates).
29. **Pareto Analysis (The 80/20 Rule):** Use window functions to find the top products that make up the top 80% of the company's total historical revenue.
30. **Order Lifecycle Bottlenecks:** Calculate the moving average time (using ROWS BETWEEN) it takes to process and ship orders (order_date to shipped_date) over the last 10 orders handled by each store to identify operational slowdowns.